K. K. Wagh Institute of Engineering
Education and Research, Nashik

Department of Computer Engineering



Simulating a Convolutional Neural Network on a Single Image

A Layer-by-Layer Study of the Matrix Computations using the Google Quick, Draw! Dataset

Submitted by

Hrishikesh Gavai

Class	BE-COMP-A
Roll Number	17
Department	Computer Engineering
Institute	KKWIEER, Nashik

Academic Year 2026–27

Contents

1 Introduction **1**
- 1.1 Purpose of this Work . . . 1
- 1.2 Layers of the Network . . . 2

2 The Quick, Draw! Dataset **3**
- 2.1 About Quick, Draw! . . . 3
- 2.2 Dataset Specifications . . . 4
- 2.3 Category Used: Van . . . 5

3 Implementation: Layer-by-Layer Simulation **6**
- 3.1 Obtaining and Loading the Dataset . . . 6
- 3.2 Viewing Sample Drawings . . . 7
- 3.3 Selecting the Input Image . . . 8
- 3.4 Normalisation . . . 10
- 3.5 Padding . . . 11
- 3.6 Kernel and Stride . . . 12
- 3.7 Convolution (Conv2D) . . . 13
  - 3.7.1 Verifying One Value by Hand . . . 14
  - 3.7.2 Visualising the Feature Map . . . 15
- 3.8 ReLU Activation . . . 16
  - 3.8.1 Visualising the ReLU Output . . . 17
- 3.9 Max Pooling . . . 18
  - 3.9.1 Visualising the Pooled Output . . . 19
- 3.10 Flatten . . . 20
- 3.11 Fully Connected Layer . . . 21
- 3.12 Softmax and Prediction . . . 22
- 3.13 Original Image and Most Reduced Image . . . 23

4 Results and Discussion **24**
- 4.1 Progression of the Data through the Network . . . 24
- 4.2 Interpretation of the Output . . . 25
- 4.3 Random Weights and Trained Weights . . . 26
- 4.4 Limitations . . . 27

5 Conclusion **28**

References **29**

List of Figures

2.1 The Quick, Draw! Dataset by Google . . . 3

3.1 The first 10 drawings in `van.npy` . . . 7
3.2 Input image: van #7 (28×28) . . . 9
3.3 Input image and the feature map after convolution . . . 15
3.4 Input image, feature map, and the result after ReLU . . . 17
3.5 ReLU output (28×28) and max-pooled output (14×14) . . . 19
3.6 Original image (28×28) and the most reduced image (14×14) . . . 23

List of Tables

1.1 Layers of the simulated CNN . . . 2

2.1 Specifications of the Quick, Draw! dataset . . . 4

4.1 Shape and number of values after each stage . . . 24
4.2 Comparison of random and trained weights . . . 26

List of Code Listings

3.1 Downloading the van drawings . . . 6
3.2 Loading the dataset . . . 6
3.3 Displaying the first 10 vans . . . 7
3.4 Selecting and saving van #7 . . . 8
3.5 Displaying the input image and its matrix . . . 9
3.6 Normalising the input to the range 0–1 . . . 10
3.7 Adding zero padding . . . 11
3.8 Defining the kernel and stride . . . 12
3.9 Convolution performed manually . . . 13
3.10 Checking the value at position (6, 10) . . . 14
3.11 Input and feature map side by side . . . 15
3.12 Applying ReLU . . . 16
3.13 Input, feature map and ReLU output . . . 17
3.14 Max pooling performed manually . . . 18
3.15 ReLU output and pooled output . . . 19
3.16 Flattening the pooled matrix . . . 20
3.17 Initialising weights and biases . . . 21
3.18 Computing the class scores manually . . . 21
3.19 Softmax and the final prediction . . . 22
3.20 Original image and pooled image . . . 23

Introduction

1.1 Purpose of this Work

A **Convolutional Neural Network** (CNN) is a neural network built for data arranged as a grid, most commonly images. It scans the image with small filters to detect local patterns such as edges, reduces the size of the data in stages, and finally assigns the image to one of a fixed set of classes.

In practice, a CNN is used through a library, and the intermediate values remain hidden. This work opens up that process. A single 28×28 drawing of a **van**, taken from Google's Quick, Draw! dataset, is passed through each layer of a small CNN *one layer at a time*. After every layer the resulting matrix is printed, displayed as an image and, where useful, checked by hand. All operations are written directly in NumPy with explicit loops, so that every multiplication, addition and comparison appears in the code itself.

The network is deliberately left **untrained**. The convolution kernel is chosen by hand and the weights of the fully connected layer are random. The arithmetic at every step is therefore exactly what a real CNN performs, but the final prediction should be read as a demonstration of the procedure, not as genuine recognition. Section 4.3 discusses this point in detail.

Tools

- **Google Colab**, an online Python notebook environment.
- **Python 3** with `NumPy` (matrix operations), `Matplotlib` (plotting), `Pillow` (saving the image as PNG) and `SciPy` (softmax).

1.2 Layers of the Network

Table 1.1 lists the stages through which the image passes, in order, with a one-line description of each and the shape of the data it produces.

Table 1.1: *Layers of the simulated CNN*

Layer	Description	Output shape
Input image	The drawing as a grid of pixel values, scaled from 0–255 to 0–1.	28×28
Padding	A border of zeros added so that the kernel can also be centred on edge pixels.	30×30
Kernel	A 3×3 grid of weights that responds to one pattern; here, vertical edges.	3×3
Stride	The number of pixels the kernel moves at each step; here, 1.	–
Conv2D	The kernel slides over the image; at each position the overlapping values are multiplied and summed.	28×28
Feature map	The output of the convolution: large values where the pattern is present.	28×28
ReLU	Every negative value is set to 0; positive values are kept.	28×28
Max pooling	Only the largest value of each 2×2 block is kept, halving each dimension.	14×14
Flatten	The 14×14 grid is written out as a single list of numbers.	196
Fully connected	All 196 values are combined with weights into one score per class.	3
Softmax	The three scores are converted into probabilities that sum to 1.	3

The Quick, Draw! Dataset

2.1 About Quick, Draw!

Quick, Draw! is an online game developed by **Google Creative Lab**. The player is given the name of an object, for example "van", and has **20 seconds** to draw it while a neural network attempts to identify the drawing. Drawings contributed by players around the world were collected and released publicly as the **Quick, Draw! Dataset**, which is today one of the largest freely available collections of hand-drawn sketches for machine learning research.



Figure 2.1: *The Quick, Draw! Dataset by Google*

Dataset page: https://quickdraw.withgoogle.com/data

2.2 Dataset Specifications

Table 2.1: *Specifications of the Quick, Draw! dataset*

Property	Details
Creator	Google Creative Lab
Total drawings	About 50 million
Categories	345 object categories (van, cat, bicycle, clock, . . .)
Source	Players of the Quick, Draw! game, worldwide
Available formats	Raw (`.ndjson`), Simplified (`.ndjson`), Binary (`.bin`), NumPy bitmap (`.npy`)
Format used here	NumPy bitmap (`.npy`), one file per category
Image size	28×28 pixels, grayscale (one channel)
Pixel values	Integers from 0 (black background) to 255 (white stroke)
Storage shape	$(N, 784)$; each row is one image flattened into 784 values
Licence	Creative Commons Attribution 4.0 International (CC BY 4.0)

Data Formats

- **Raw data**: each drawing stored as pen strokes with exact coordinates and timing, together with metadata such as the category, the player's country and whether the game recognised it.
- **Simplified data**: the same strokes after cleaning, alignment and scaling, with timing removed.
- **Binary data**: the simplified data in a compact binary encoding.
- **NumPy bitmaps**: the simplified drawings rendered as 28×28 grayscale images and stored as `.npy` arrays. This is the format used here, since a CNN operates on pixel grids rather than on strokes.

2.3 Category Used: Van

Of the 345 categories, this work uses **van**, stored in the file `van.npy`. The file holds **165,909** drawings, each stored as a row of 784 pixel values (28×28). Drawing number **7** was selected as the single input image.

The van was chosen because its outline contains clear *vertical* strokes (the rear of the body, the front pillar and the sides of the wheels). These respond strongly to a vertical edge detector, which makes the effect of the convolution easy to observe.

Links

- Dataset page: https://quickdraw.withgoogle.com/data
- Dataset repository: https://github.com/googlecreativelab/quickdraw-dataset
- File used: https://storage.googleapis.com/quickdraw_dataset/full/numpy_bitmap/van.npy

Implementation: Layer-by-Layer Simulation

3.1 Obtaining and Loading the Dataset

The van.npy file is downloaded into the Colab session and loaded as a NumPy array.

• • • In [3.1]: Code 3.1 - Downloading the van drawings

```
# Download the van drawings from Google's Quick Draw dataset
!wget -q "https://storage.googleapis.com/quickdraw_dataset/full/numpy_bitmap/van.npy" -O
    van.npy
```

Explanation. The leading ! runs a shell command from inside the notebook. wget fetches van.npy from Google's public storage and saves it in the current session. The option -q suppresses progress messages, so the cell produces no output.

• • • In [3.2]: Code 3.2 - Loading the dataset

```
import numpy as np                  # for working with numbers and arrays
import matplotlib.pyplot as plt     # for showing images
from PIL import Image               # for saving the image as a PNG

# Load all van drawings
data = np.load("van.npy")

# Rows = number of drawings, columns = 784 pixels per drawing
print("Shape:", data.shape)
```

• • • Out: Output

```
Shape: (165909, 784)
```

Explanation. The file is read into a single array of 165,909 rows and 784 columns. Each row is one drawing, and each column holds the brightness of one pixel, since $28 \times 28 = 784$.

3.2 Viewing Sample Drawings

RESHAPING A ROW INTO AN IMAGE

$$\underbrace{(x_0, x_1, \ldots, x_{783})}_{\text{784 values}} \longrightarrow \underbrace{X_{28\times 28}}_{\text{image}}, \qquad X(i,j) = x_{28i+j}$$

In [3.3]: Code 3.3 - Displaying the first 10 vans

```
plt.figure(figsize=(10, 4))

for i in range(10):
    img = data[i].reshape(28, 28)     # turn 784 numbers back into a 28x28 image

    plt.subplot(2, 5, i + 1)          # 2 rows, 5 columns, position i+1
    plt.imshow(img, cmap="gray")      # show it in black and white
    plt.title("van " + str(i))
    plt.axis("off")                   # hide the axis numbers

plt.show()
```

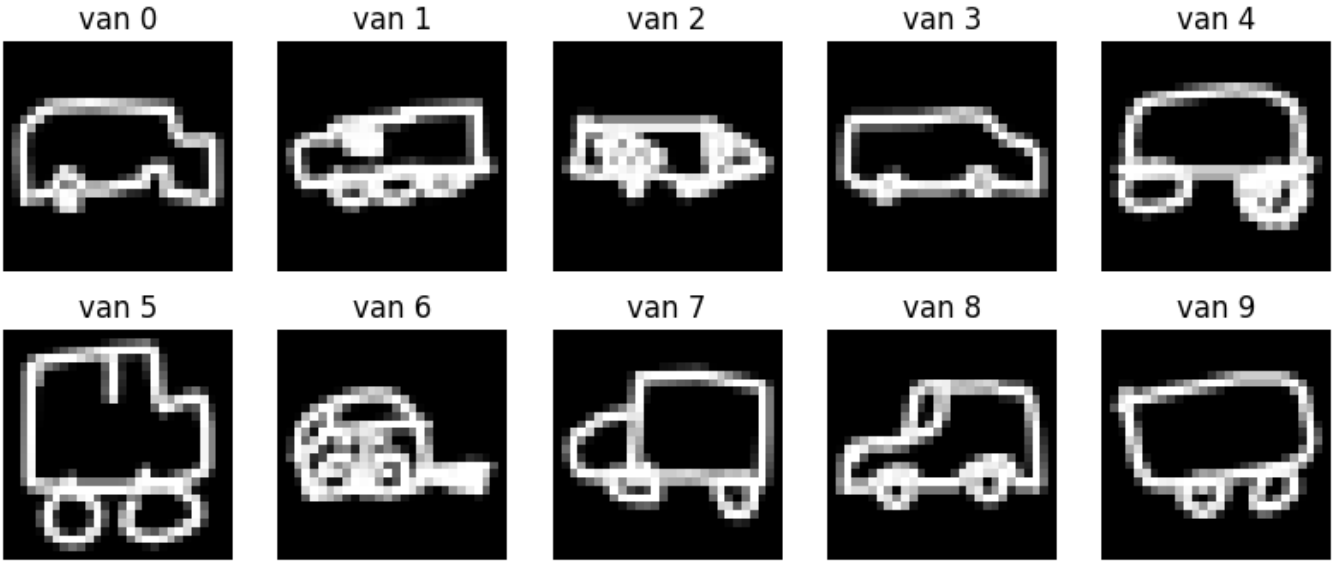


Figure 3.1: *The first 10 drawings in* `van.npy`

Explanation. The loop takes drawings 0 to 9, rebuilds each row of 784 values into a 28×28 grid, and places the results in a grid of two rows and five columns. The strokes appear white on a black background because that is how the dataset stores them.

3.3 Selecting the Input Image

• • • In [3.4]: Code 3.4 - Selecting and saving van #7

```
idx = 7                                # the drawing we want to use

img = data[idx].reshape(28, 28)        # 784 numbers -> 28x28 image

Image.fromarray(img).save("van.png")   # save it as a PNG file

print("Shape:", img.shape)             # should be (28, 28)
print("Smallest value:", img.min())    # 0 = black background
print("Largest value:", img.max())     # 255 = white stroke
```

• • • Out: Output

```
Shape: (28, 28)
Smallest value: 0
Largest value: 255
```

Explanation. Drawing number 7 is used for the rest of the simulation. It is reshaped into a 28×28 matrix and also saved as `van.png`. Its values span the full range from 0 (background) to 255 (stroke).

• • • In [3.5]: Code 3.5 - Displaying the input image and its matrix

```
# Show the image bigger
plt.figure(figsize=(3, 3))
plt.imshow(img, cmap="gray")
plt.title("van " + str(idx))
plt.axis("off")
plt.show()

# Print all 28x28 numbers on one screen without wrapping
print("\n")
np.set_printoptions(linewidth=250)
print(img)
```

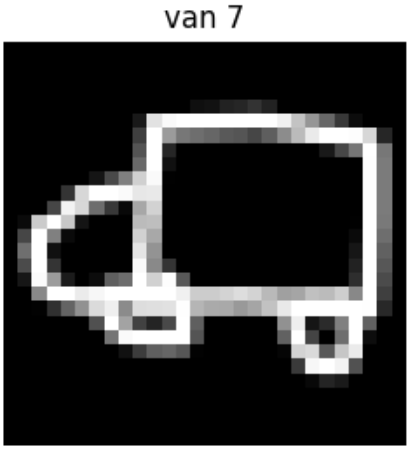


Figure 3.2: *Input image: van #7 (*28×28*)*

• • • Out: Output

```
[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0  15  18  34  38  51  34   1   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0  25 226 250 255 255 255 255 255 255 255 241 196 150 103  56  12   0   0]
 [  0   0   0   0   0   0   0   0   0  63 255 155 119 110 102  90  85  70  94 140 187 234 255 255 253 215  38   0]
 [  0   0   0   0   0   0   0   0   0  84 255  38   0   0   0   0   0   0   0   0   0   0  25  71 118 255 119   0]
 [  0   0   0   0   0   0   0   0   0 106 255  16   0   0   0   0   0   0   0   0   0   0   0   0   0 254 123   0]
 [  0   0   0   0   0   0   0  21  74 127 208 250   1   0   0   0   0   0   0   0   0   0   0   0   0 254 123   0]
 [  0   0   0   0  52 215 255 255 250 231 233   0   0   0   0   0   0   0   0   0   0   0   0   0   0 254 123   0]
 [  0   0   0  70 243 216 109  56   8 167 210   0   0   0   0   0   0   0   0   0   0   0   0   0   0 254 123   0]
 [  0  10 203 255 200  20   0   0   0 187 190   0   0   0   0   0   0   0   0   0   0   0   0   0   1 254 121   0]
 [  0  70 255 128  10   0   0   0   0 209 168   0   0   0   0   0   0   0   0   0   0   0   0   0  10 255 110   0]
 [  0 111 255  16   0   0   0   0   0 229 148   0   0   0   0   0   0   0   0   0   0   0   0   0  22 255  98   0]
 [  0  75 255  52   0   0   0   0   0 249 137  53   0   0   0   0   0   0   0   0   0   0   0   0  34 255  87   0]
 [  0  22 251 156  27   0  12  95 109 255 209 253  88   0   0   0   0   0   0   0   0   0   0   0  46 255  75   0]
 [  0   0 136 255 255 235 247 255 255 255 255 255 240 204 204 212 221 212 185 160 165 192 187 206 136 255  63   0]
 [  0   0   2  49 100 145 219 239 139 221 223 225 254 181 170 169 153 165 194 233 255 255 255 255 254  40   0]
 [  0   0   0   0   0   0  80 255 163  66  36  74 255  95   0   0   0   0   0 145 238  21  43 128 255  97   0   0]
 [  0   0   0   0   0   0   0 117 250 255 255 255 255  84   0   0   0   0   0  97 255  68   0 110 255  13   0   0]
 [  0   0   0   0   0   0   0   0  17  60  95 102  86   2   0   0   0   0   0   5 211 218  88 198 235   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  58 247 255 244  82   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   8  36  26   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]
```

Explanation. This 28×28 matrix is the **input matrix**. A value of 0 is background and 255 is a fully drawn stroke; intermediate values are the soft edges left when the drawing was reduced to 28×28. The outline of the van can be traced in the numbers: the roof along rows 5–6, the rear of the body down column 25, and the two wheels in rows 19–23.

3.4 Normalisation

NORMALISATION

$$x_{\text{norm}} = \frac{x}{255}, \qquad 0 \le x_{\text{norm}} \le 1$$

$$255 \to 1.00, \qquad 226 \to 0.89, \qquad 0 \to 0.00$$

• • • In [3.6]: Code 3.6 - Normalising the input to the range 0-1

```
# Turn 0-255 values into 0-1 values
image = img / 255.0

np.set_printoptions(linewidth=250, precision=2, suppress=True) # show 2 decimal places

print("Input shape:", image.shape)
print("\n")
print(image)
```

• • • Out: Output

```
Input shape: (28, 28)


[[0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0. ]
 [
```

3.5 Padding

SIZE AFTER PADDING

$$N_{\text{padded}} = N + 2P = 28 + 2(1) = 30$$

• • • In [3.7]: Code 3.7 - Adding zero padding

```
padding = 1

# Add a border of zeros around the image
padded = np.pad(image, padding)

print("Padded shape:", padded.shape)     # 28x28 becomes 30x30
print("\n")
print(padded)
```

• • • Out: Output

```
Padded shape: (30, 30)


[[0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.06 0.07 0.13 0.15 0.2  0.13 0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.1  0.89 0.98 1.   1.   1.   1.   1.   1.   1.   0.95 0.77 0.59 0.4  0.22 0.05 0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.25 1.   0.61 0.47 0.43 0.4  0.35 0.33 0.27 0.37 0.55 0.73 0.92 1.   1.   0.99 0.84 0.15 0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.33 1.   0.15 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.1  0.28 0.46 1.   0.47 0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.42 1.   0.06 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   1.   0.48 0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.08 0.29 0.5  0.82 0.98 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   1.   0.48 0.   0.  ]
 [0.   0.   0.   0.   0.   0.2  0.84 1.   1.   0.98 0.91 0.91 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   1.   0.48 0.   0.  ]
 [0.   0.   0.   0.   0.27 0.95 0.85 0.43 0.22 0.03 0.65 0.82 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   1.   0.48 0.   0.  ]
 [0.   0.   0.04 0.8  1.   0.78 0.08 0.   0.   0.   0.73 0.75 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   1.   0.47 0.   0.  ]
 [0.   0.   0.27 1.   0.5  0.04 0.   0.   0.   0.   0.82 0.66 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.04 1.   0.43 0.   0.  ]
 [0.   0.   0.44 1.   0.06 0.   0.   0.   0.   0.   0.9  0.58 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.09 1.   0.38 0.   0.  ]
 [0.   0.   0.29 1.   0.2  0.   0.   0.   0.   0.   0.98 0.54 0.21 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.13 1.   0.34 0.   0.  ]
 [0.   0.   0.09 0.98 0.61 0.11 0.   0.05 0.37 0.43 1.   0.82 0.99 0.35 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.18 1.   0.29 0.   0.  ]
 [0.   0.   0.   0.53 1.   1.   0.92 0.97 1.   1.   1.   1.   1.   0.94 0.8  0.8  0.83 0.87 0.83 0.73 0.63 0.65 0.75 0.73 0.81 0.53 1.   0.25 0.   0.  ]
 [0.   0.   0.   0.01 0.19 0.39 0.57 0.86 0.94 0.55 0.87 0.87 0.88 1.   0.71 0.67 0.66 0.6  0.65 0.76 0.91 1.   1.   1.   1.   1.   1.   0.16 0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.31 1.   0.64 0.26 0.14 0.29 1.   0.37 0.   0.   0.   0.   0.   0.57 0.93 0.08 0.17 0.5  1.   0.38 0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.46 0.98 1.   1.   1.   1.   0.33 0.   0.   0.   0.   0.   0.38 1.   0.27 0.   0.43 1.   0.05 0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.07 0.24 0.37 0.4  0.34 0.01 0.   0.   0.   0.   0.   0.02 0.83 0.85 0.35 0.78 0.92 0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.23 0.97 1.   0.96 0.32 0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.03 0.14 0.1  0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]]
```

Explanation. A border one pixel wide, filled with zeros, is added on all four sides, enlarging the image from 28×28 to 30×30. Without it, a 3×3 kernel could not be centred on the outermost pixels and the output would shrink to 26×26. With it, the output of the convolution keeps the original size.

3.6 Kernel and Stride

VERTICAL EDGE KERNEL AND STRIDE

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \qquad S = 1$$

• • • In [3.8]: Code 3.8 - Defining the kernel and stride

```
# 3x3 vertical edge detector:
# left column negative, right column positive
kernel = np.array([[-1, 0, 1],
                   [-1, 0, 1],
                   [-1, 0, 1]])

stride = 1

print("Kernel:")
print(kernel)
print("\nStride:", stride)
```

• • • Out: Output

```
Kernel:
[[-1  0  1]
 [-1  0  1]
 [-1  0  1]]

Stride: 1
```

Explanation. The (Prewitt) kernel subtracts the left column of each 3×3 window from its right column. Where the brightness changes from left to right, as it does across a vertical stroke, the result is large; in flat regions the two columns cancel and the result is close to zero. A stride of 1 moves the kernel one pixel at a time, so that every position of the image is examined.

3.7 Convolution (Conv2D)

OUTPUT SIZE AND CONVOLUTION

$$O = \left\lfloor \frac{N - K + 2P}{S} \right\rfloor + 1 = \frac{28 - 3 + 2(1)}{1} + 1 = 28$$

• • • In [3.9]: Code 3.9 - Convolution performed manually

```
k = 3                                                  # kernel size
out_size = (28 - k + 2 * padding) // stride + 1        # output size formula
print("Output size:", out_size, "x", out_size)

feature_map = np.zeros((out_size, out_size))           # empty matrix to fill

for i in range(out_size):                              # go down row by row
    for j in range(out_size):                          # go across column by column
        r = i * stride                                 # top-left row of the window
        c = j * stride                                 # top-left column of the window

        patch = padded[r:r+3, c:c+3]                   # take a 3x3 piece of the image

        feature_map[i, j] = np.sum(patch * kernel)   # multiply and add up

print("\n")
print(feature_map)
```

• • • Out: Output

```
Output size: 28 x 28


[[ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.06  0.07  0.07  0.08  0.07 -0.02 -0.2  -0.13 -0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.1   0.89  0.88  0.11  0.08  0.07  0.07  0.08  0.07 -0.02 -0.25 -0.36 -0.36 -0.36 -0.37 -0.36 -0.22 -0.05  0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.35  1.89  1.24 -0.42 -0.1   0.   -0.    0.01 -0.01  0.02  0.02  0.    0.01 -0.1  -0.29 -0.36 -0.38 -0.89 -0.84 -0.15]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.67  2.89  1.06 -1.42 -0.31 -0.07 -0.08 -0.07 -0.08  0.04  0.22  0.13  0.01 -0.   -0.01  0.    0.35 -0.89 -1.84 -0.62]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.99  3.   -0.17 -2.53 -0.39 -0.07 -0.08 -0.07 -0.08  0.04  0.27  0.36  0.37  0.36  0.36  0.36  1.56 -0.36 -2.84 -1.1 ]
 [ 0.    0.    0.    0.    0.    0.08  0.29  0.42  1.27  2.48 -1.35 -2.98 -0.22  0.    0.    0.    0.    0.    0.    0.    0.    0.1   0.28  0.36  2.71  0.97 -2.99 -1.43]
 [ 0.    0.    0.    0.2   0.84  0.88  0.45  0.4   0.85  1.42 -2.07 -2.89 -0.07  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    2.99  1.45 -2.99 -1.45]
 [ 0.    0.    0.27  1.16  1.42  0.35 -0.18  0.    0.87  1.21 -2.37 -2.72 -0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    2.99  1.45 -2.99 -1.45]
 [ 0.04  0.8   1.24  1.15  0.49 -0.51 -0.55 -0.42  1.07  1.47 -2.29 -2.48  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    2.99  1.44 -2.99 -1.44]
 [ 0.31  1.8   1.46 -0.02 -0.85 -1.35 -0.71 -0.4   1.99  2.2  -2.21 -2.23  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.04  2.99  1.35 -2.99 -1.39]
 [ 0.75  2.8   0.82 -1.97 -1.49 -0.82 -0.08  0.    2.45  1.98 -2.45 -1.98  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.13  3.    1.16 -3.   -1.29]
 [ 1.    3.   -0.24 -2.96 -0.77 -0.04  0.    0.    2.69  1.78 -2.49 -1.78 -0.21  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.26  3.    0.9  -3.   -1.16]
 [ 0.82  2.98  0.06 -2.88 -0.88 -0.06  0.37  0.38  2.5   1.51 -1.67 -1.59 -1.2  -0.35  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.4   3.    0.62 -3.   -1.02]
 [ 0.38  2.52  1.44 -1.41 -0.89 -0.09  0.45  0.41  1.6   0.93 -0.78 -1.07 -1.4  -0.49  0.03  0.07  0.   -0.14 -0.2  -0.08  0.13  0.09  0.05  0.11  2.19  0.04 -3.   -0.88]
 [ 0.09  1.53  1.72 -0.03 -0.31  0.38  0.82  0.1   0.56  0.72  0.01 -0.41 -1.36 -0.82 -0.02  0.   -0.02  0.02  0.06  0.16  0.21  0.09  0.05 -0.02  1.19 -1.02 -3.   -0.7 ]
 [ 0.    0.54  1.19  0.85  0.3   0.75  1.45  0.04 -0.81 -0.17  0.05  0.92 -0.29 -1.47 -0.39  0.   -0.02  0.02  0.63  1.09 -0.27 -0.68  0.47  0.63  0.07 -2.13 -2.38 -0.4 ]
 [ 0.    0.01  0.19  0.38  0.38  0.78  1.83  0.99 -0.27 -0.15  0.05  0.98 -0.76 -2.33 -0.75 -0.07 -0.02  0.16  1.22  2.17 -0.51 -1.76  0.58  1.83 -0.51 -2.84 -1.43 -0.16]
 [ 0.    0.    0.    0.    0.    0.31  1.46  1.37  0.04 -0.17  0.2   0.82 -0.98 -2.34 -0.71  0.    0.    0.    0.97  2.76  0.24 -2.25  0.51  2.41 -1.28 -2.92 -0.43  0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.46  1.05  0.78  0.33  0.16 -0.04 -1.06 -1.34 -0.34  0.    0.    0.    0.4   2.05  1.69 -0.71  0.07  0.9  -2.11 -2.24 -0.05  0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.07  0.24  0.31  0.16 -0.04 -0.39 -0.34 -0.01  0.    0.    0.    0.02  1.05  1.84  0.43 -0.02 -0.24 -1.84 -1.24  0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.23  1.    0.91  0.06 -0.82 -1.06 -0.32  0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.03  0.14  0.07 -0.14 -0.1   0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.  ]]
```

Explanation. The two loops visit all $28 \times 28 = 784$ output positions. At each position a 3×3 patch is cut from the padded image, multiplied element by element with the kernel, and the nine products are summed to give one entry of the **feature map**.

3.7.1 Verifying One Value by Hand

ELEMENT-WISE PRODUCT AND SUM AT POSITION (6, 10)

$$\begin{bmatrix} 0.10 & 0.89 & 0.98 \\ 0.25 & 1.00 & 0.61 \\ 0.33 & 1.00 & 0.15 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -0.10 & 0 & 0.98 \\ -0.25 & 0 & 0.61 \\ -0.33 & 0 & 0.15 \end{bmatrix}$$

$$\text{Sum} = (0.98 + 0.61 + 0.15) - (0.10 + 0.25 + 0.33) = 1.74 - 0.68 \approx 1.06$$

• • • In [3.10]: Code 3.10 - Checking the value at position (6, 10)

```
i, j = 6, 10                          # pick one position in the output

patch = padded[i:i+3, j:j+3]          # the 3x3 piece the kernel saw there

print("Image patch:")
print(patch)
print("\nKernel:")
print(kernel)
print("\nPatch x Kernel (element by element):")
print(patch * kernel)
print("\nSum =", np.sum(patch * kernel))
print("Value in feature map =", feature_map[i, j])
```

• • • Out: Output

```
Image patch:
[[0.1  0.89 0.98]
 [0.25 1.   0.61]
 [0.33 1.   0.15]]

Kernel:
[[-1  0  1]
 [-1  0  1]
 [-1  0  1]]

Patch x Kernel (element by element):
[[-0.1   0.    0.98]
 [-0.25  0.    0.61]
 [-0.33  0.    0.15]]

Sum = 1.0627450980392155
Value in feature map = 1.0627450980392155
```

Explanation. At position (6, 10) the kernel lies over the upper-left corner of the van. The sum computed by hand and the value stored in the feature map agree exactly, which confirms that a convolution consists of nothing more than multiplication and addition. The multiplication is element by element, not a matrix product.

3.7.2 Visualising the Feature Map

• • • In [3.11]: Code 3.11 - Input and feature map side by side

```
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Input (28x28)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(feature_map, cmap="gray")
plt.title("Feature map (28x28)")
plt.axis("off")

plt.show()
```

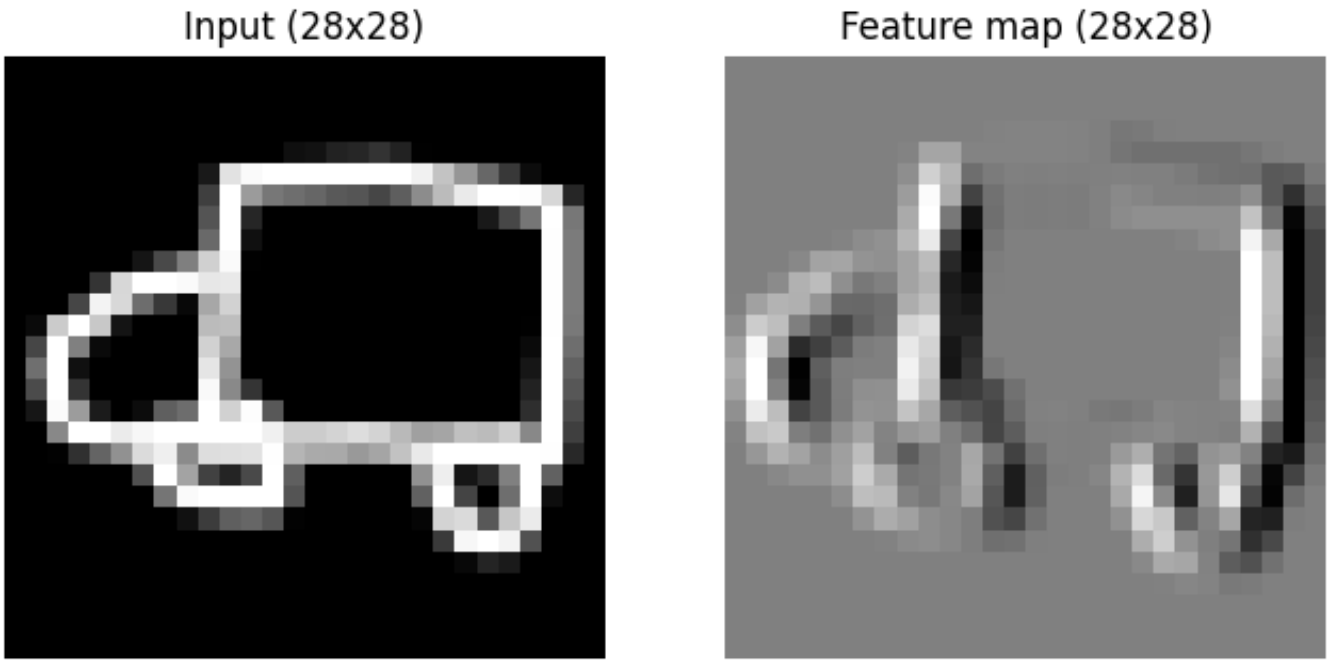


Figure 3.3: *Input image and the feature map after convolution*

Explanation. In the feature map, grey represents 0, white represents positive values and black represents negative values. Every vertical stroke of the van therefore appears as a white band beside a black band, which gives the image an embossed appearance. The roof and the lower edge of the body almost vanish, because the kernel responds only to change in the horizontal direction.

3.8 ReLU Activation

RECTIFIED LINEAR UNIT

$$\mathbf{ReLU}(x) = \mathbf{max}(0, x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

$$\mathbf{2.89 \to 2.89}, \qquad \mathbf{-1.42 \to 0}$$

• • • In [3.12]: Code 3.12 - Applying ReLU

```
# ReLU: negative values become 0, positive values stay the same
relu_map = np.maximum(0, feature_map)

print("ReLU output shape:", relu_map.shape)     # still 28x28
print("\n")
print(relu_map)
```

• • • Out: Output

```
ReLU output shape: (28, 28)


[[0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.06 0.07 0.07 0.08 0.07 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.08 0.11 0.08 0.07 0.07 0.08 0.07 0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.1  0.89 0.88 0.11 0.08 0.07 0.07 0.08 0.07 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.35 1.89 1.24 0.   0.   0.   0.   0.01 0.   0.02 0.02 0.   0.01 0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.67 2.89 1.06 0.   0.   0.   0.   0.   0.   0.04 0.22 0.13 0.01 0.   0.   0.   0.35 0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.99 3.   0.   0.   0.   0.   0.   0.   0.   0.04 0.27 0.36 0.37 0.36 0.36 0.36 1.56 0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.08 0.29 0.42 1.27 2.48 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.1  0.28 0.36 2.71 0.97 0.   0.  ]
 [0.   0.   0.   0.2  0.84 0.88 0.45 0.4  0.85 1.42 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   2.99 1.45 0.   0.  ]
 [0.   0.   0.27 1.16 1.42 0.35 0.   0.   0.87 1.21 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   2.99 1.45 0.   0.  ]
 [0.04 0.8  1.24 1.15 0.49 0.   0.   0.   1.07 1.47 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   2.99 1.44 0.   0.  ]
 [0.31 1.8  1.46 0.   0.   0.   0.   0.   1.99 2.2  0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.04 2.99 1.35 0.   0.  ]
 [0.75 2.8  0.82 0.   0.   0.   0.   0.   2.45 1.98 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.13 3.   1.16 0.   0.  ]
 [1.   3.   0.   0.   0.   0.   0.   0.   2.69 1.78 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.26 3.   0.9  0.   0.  ]
 [0.82 2.98 0.06 0.   0.   0.   0.37 0.38 2.5  1.51 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.4  3.   0.62 0.   0.  ]
 [0.38 2.52 1.44 0.   0.   0.   0.45 0.41 1.6  0.93 0.   0.   0.   0.   0.03 0.07 0.   0.   0.   0.   0.13 0.09 0.05 0.11 2.19 0.04 0.   0.  ]
 [0.09 1.53 1.72 0.   0.   0.38 0.82 0.1  0.56 0.72 0.01 0.   0.   0.   0.   0.   0.   0.02 0.06 0.16 0.21 0.09 0.05 0.   1.19 0.   0.   0.  ]
 [0.   0.54 1.19 0.85 0.3  0.75 1.45 0.04 0.   0.   0.05 0.92 0.   0.   0.   0.   0.   0.02 0.63 1.09 0.   0.   0.47 0.63 0.07 0.   0.   0.  ]
 [0.   0.01 0.19 0.38 0.38 0.78 1.83 0.99 0.   0.   0.05 0.98 0.   0.   0.   0.   0.   0.16 1.22 2.17 0.   0.   0.58 1.83 0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.31 1.46 1.37 0.04 0.   0.2  0.82 0.   0.   0.   0.   0.   0.   0.97 2.76 0.24 0.   0.51 2.41 0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.46 1.05 0.78 0.33 0.16 0.   0.   0.   0.   0.   0.   0.   0.4  2.05 1.69 0.   0.07 0.9  0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.07 0.24 0.31 0.16 0.   0.   0.   0.   0.   0.   0.   0.02 1.05 1.84 0.43 0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.23 1.   0.91 0.06 0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.03 0.14 0.07 0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]]
```

Explanation. `np.maximum(0, x)` compares every entry with 0 and keeps the larger of the two, so all negative values become 0 and positive values pass through unchanged. The shape remains 28×28. ReLU keeps only the responses that indicate the pattern is present. It also makes the network non-linear; without it, any number of stacked layers would reduce to a single linear operation.

3.8.1 Visualising the ReLU Output

In [3.13]: Code 3.13 - Input, feature map and ReLU output

```
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Input")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(feature_map, cmap="gray")
plt.title("Feature map")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(relu_map, cmap="gray")
plt.title("After ReLU")
plt.axis("off")

plt.show()
```

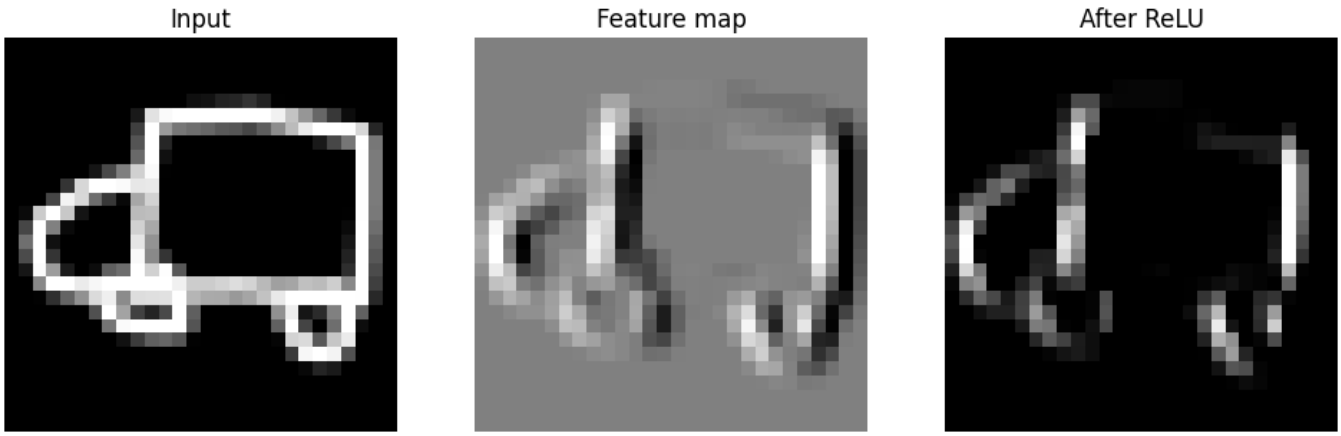


Figure 3.4: *Input image, feature map, and the result after ReLU*

Explanation. After ReLU the black regions disappear. With no negative values left, 0 becomes the darkest value, so the background is displayed as black and only the white sides of the vertical strokes remain.

3.9 Max Pooling

MAX POOLING WITH A 2×2 WINDOW AND STRIDE 2

$$P(i,j) = \max\left\{ R(2i,2j),\ R(2i,2j+1),\ R(2i+1,2j),\ R(2i+1,2j+1) \right\}$$

$$O = \frac{28-2}{2} + 1 = 14$$

In [3.14]: Code 3.14 - Max pooling performed manually

```
pool_size = 2                                          # 2x2 window
pool_stride = 2                                        # jump 2 pixels each time
pool_out = (28 - pool_size) // pool_stride + 1   # same formula as conv (no padding)
print("Pooling output size:", pool_out, "x", pool_out)

pooled = np.zeros((pool_out, pool_out))          # empty 14x14 matrix to fill

for i in range(pool_out):                              # go down row by row
    for j in range(pool_out):                          # go across column by column
        r = i * pool_stride                            # top-left row of the window
        c = j * pool_stride                            # top-left column of the window

        block = relu_map[r:r+2, c:c+2]           # take a 2x2 piece

        pooled[i, j] = np.max(block)             # keep only the biggest value

print("\n")
print(pooled)
```

Out: Output

```
Pooling output size: 14 x 14


[[0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.07 0.08 0.07 0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   1.89 1.24 0.08 0.08 0.07 0.02 0.01 0.   0.   0.  ]
 [0.   0.   0.   0.   3.   1.06 0.   0.   0.04 0.36 0.37 0.36 1.56 0.  ]
 [0.   0.2  0.88 0.45 2.48 0.   0.   0.   0.   0.   0.1  0.36 2.99 0.  ]
 [0.8  1.24 1.42 0.   1.47 0.   0.   0.   0.   0.   0.   0.   2.99 0.  ]
 [2.8  1.46 0.   0.   2.45 0.   0.   0.   0.   0.   0.   0.13 3.   0.  ]
 [3.   0.06 0.   0.38 2.69 0.   0.   0.   0.   0.   0.   0.4  3.   0.  ]
 [2.52 1.72 0.38 0.82 1.6  0.01 0.   0.07 0.02 0.16 0.21 0.11 2.19 0.  ]
 [0.54 1.19 0.78 1.83 0.   0.98 0.   0.   0.16 2.17 0.   1.83 0.07 0.  ]
 [0.   0.   0.31 1.46 0.78 0.82 0.   0.   0.   2.76 1.69 2.41 0.   0.  ]
 [0.   0.   0.   0.07 0.31 0.16 0.   0.   0.   1.05 1.84 0.06 0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.14 0.07 0.   0.  ]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.  ]]
```

Explanation. The loops have the same structure as in the convolution, with three differences: the window is 2×2, it advances two pixels at a time so the blocks do not overlap, and instead of a weighted sum it simply takes the **maximum**. Pooling has no weights. The data shrinks from 28×28 (784 values) to 14×14 (196 values), keeping the strongest responses and their approximate positions.

3.9.1 Visualising the Pooled Output

In [3.15]: Code 3.15 - ReLU output and pooled output

```
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(relu_map, cmap="gray")
plt.title("After ReLU (28x28)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(pooled, cmap="gray")
plt.title("After Max Pooling (14x14)")
plt.axis("off")

plt.show()
```

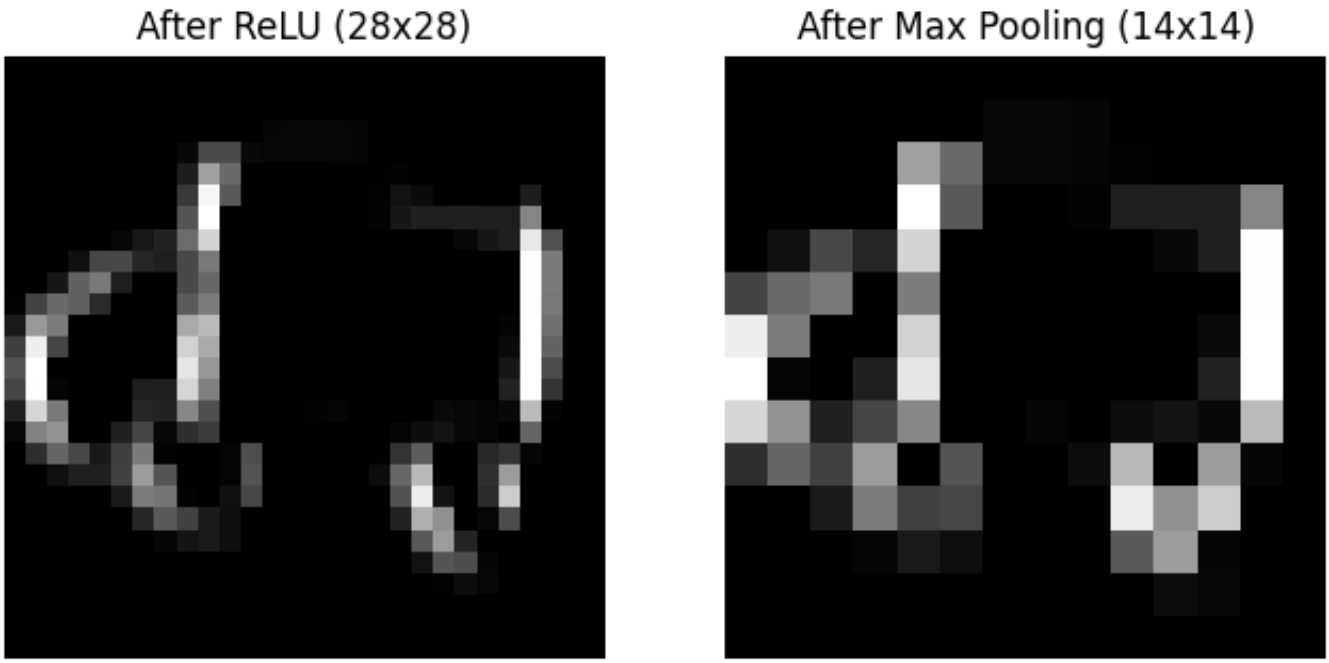


Figure 3.5: *ReLU output* (28×28) *and max-pooled output* (14×14)

Explanation. The pooled image is a coarser version of the ReLU image at half the resolution. The same edges are still present, now represented by a quarter of the original number of values.

3.10 Flatten

INDEX MAPPING

$$k = 14\,i + j, \qquad 14 \times 14 = 196 \text{ values}$$

$$(5,\,12) \;\rightarrow\; 5 \times 14 + 12 = 82$$

• • • In [3.16]: Code 3.16 - Flattening the pooled matrix

```
# Lay out all 14x14 values in one long list, row by row
flat = pooled.flatten()

print("Before flatten:", pooled.shape)    # (14, 14)
print("\nAfter flatten: ", flat.shape)      # (196,)
print("\n")
print(flat)
```

• • • Out: Output

```
Before flatten: (14, 14)

After flatten:  (196,)


[0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.07 0.08 0.07 0.   0.   0.   0.   0.   0.   0.   0.
0.   1.89 1.24 0.08 0.08 0.07 0.02 0.01 0.   0.   0.   0.   0.   0.   0.   0.   3.   1.06 0.
 0.   0.04 0.36 0.37 0.36 1.56 0.   0.   0.2  0.88 0.45 2.48 0.   0.   0.   0.   0.   0.1  0.36 2.99 0.   0.8  1.24 1.42 0.   1.47 0.   0.   0.   0.   0.
0.   0.   2.99 0.   2.8  1.46 0.   0.   2.45 0.   0.   0.   0.   0.   0.   0.13 3.   0.
 3.   0.06 0.   0.38 2.69 0.   0.   0.   0.   0.   0.   0.4  3.   0.   2.52 1.72 0.38 0.82 1.6  0.01 0.   0.07 0.02 0.16 0.21 0.11 2.19 0.   0.54 1.19 0.78
1.83 0.   0.98 0.   0.   0.16 2.17 0.   1.83 0.07 0.   0.   0.   0.31 1.46 0.78 0.82 0.
 0.   0.   2.76 1.69 2.41 0.   0.   0.   0.   0.   0.07 0.31 0.16 0.   0.   0.   1.05 1.84 0.06 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.
0.14 0.07 0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0. ]
```

Explanation. Flattening involves no arithmetic. The 196 values are only rearranged, row after row, into a single list (the long lines are wrapped here to fit the page). This step links the part of the network that works on images (convolution, ReLU and pooling) with the part that makes the decision (the fully connected layer), which accepts a plain list as input.

3.11 Fully Connected Layer

WEIGHTS, BIASES AND CLASS SCORES

$$W \in \mathbb{R}^{196\times 3} \ \ (\textbf{588 weights}), \qquad b \in \mathbb{R}^{3}$$

$$z_k = \sum_{n=0}^{195} x_n\, w_{n,k} \ + \ b_k, \qquad k \in \{\textbf{van, car, bus}\}$$

• • • In [3.17]: Code 3.17 - Initialising weights and biases

```
np.random.seed(0)                          # same random numbers every time you run it

classes = ["van", "car", "bus"]

weights = np.random.randn(196, 3) * 0.1    # 196 inputs x 3 classes = 588 weights
biases = np.zeros(3)                       # one bias per class, set to 0

print("Weights shape:", weights.shape)     # (196, 3)
print("Biases:", biases)
```

• • • Out: Output

```
Weights shape: (196, 3)
Biases: [0. 0. 0.]
```

Explanation. Three example classes are defined: van, car and bus. Because the network is not trained, its 588 weights are filled with small random numbers, scaled by 0.1 to keep the scores moderate. Fixing the seed with `np.random.seed(0)` produces the same numbers on every run, so the results can be reproduced. All three biases are set to 0.

• • • In [3.18]: Code 3.18 - Computing the class scores manually

```
scores = np.zeros(3)                       # empty list for the 3 scores

for k in range(3):                         # for each class
    total = 0
    for n in range(196):                   # go through all 196 inputs
        total = total + flat[n] * weights[n, k]   # input x its weight, keep adding
    scores[k] = total + biases[k]          # add the bias at the end

print("Scores:", scores)
```

• • • Out: Output

```
Scores: [ 0.36  0.09 -0.71]
```

Explanation. The outer loop runs over the three classes. The inner loop multiplies each of the 196 inputs by its weight for that class and accumulates the products; the bias is added once, after the loop. In total $196 \times 3 = 588$ multiplications are carried out. The resulting raw scores, called *logits*, may take any positive or negative value.

3.12 Softmax and Prediction

SOFTMAX

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad \textbf{Prediction} = \arg\max_k \; p_k$$

$$e^{0.36} \approx 1.43, \qquad e^{0.09} \approx 1.09, \qquad e^{-0.71} \approx 0.49, \qquad \sum \approx 3.02$$

$$p_{\text{van}} \approx \frac{1.43}{3.02} \approx 0.47, \qquad p_{\text{car}} \approx \frac{1.09}{3.02} \approx 0.36, \qquad p_{\text{bus}} \approx \frac{0.49}{3.02} \approx 0.16$$

• • • In [3.19]: Code 3.19 - Softmax and the final prediction

```
from scipy.special import softmax

probs = softmax(scores)                    # does exp, sum, and divide in one step

print("Scores:       ", scores)
print("Probabilities:", probs)
print("Sum:", np.sum(probs))               # should be 1.0

best = np.argmax(probs)                    # position of the highest probability

print("\n")
for k in range(3):
    print(classes[k], ":", round(probs[k] * 100, 2), "%")

print("\nPrediction:", classes[best])
```

• • • Out: Output

```
Scores:        [ 0.36  0.09 -0.71]
Probabilities: [0.47 0.36 0.16]
Sum: 1.0


van : 47.4 %
car : 36.33 %
bus : 16.27 %

Prediction: van
```

Explanation. `softmax` raises e to the power of each score, which makes every value positive, and then divides each result by their total, so that the probabilities sum to exactly 1. `np.argmax` returns the position of the largest probability. The network assigns 47.4% to van, 36.33% to car and 16.27% to bus, and therefore predicts **van**.

3.13 Original Image and Most Reduced Image

In [3.20]: Code 3.20 - Original image and pooled image

```
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original (28x28)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(pooled, cmap="gray")
plt.title("After Max Pooling (14x14)")
plt.axis("off")

plt.show()
```

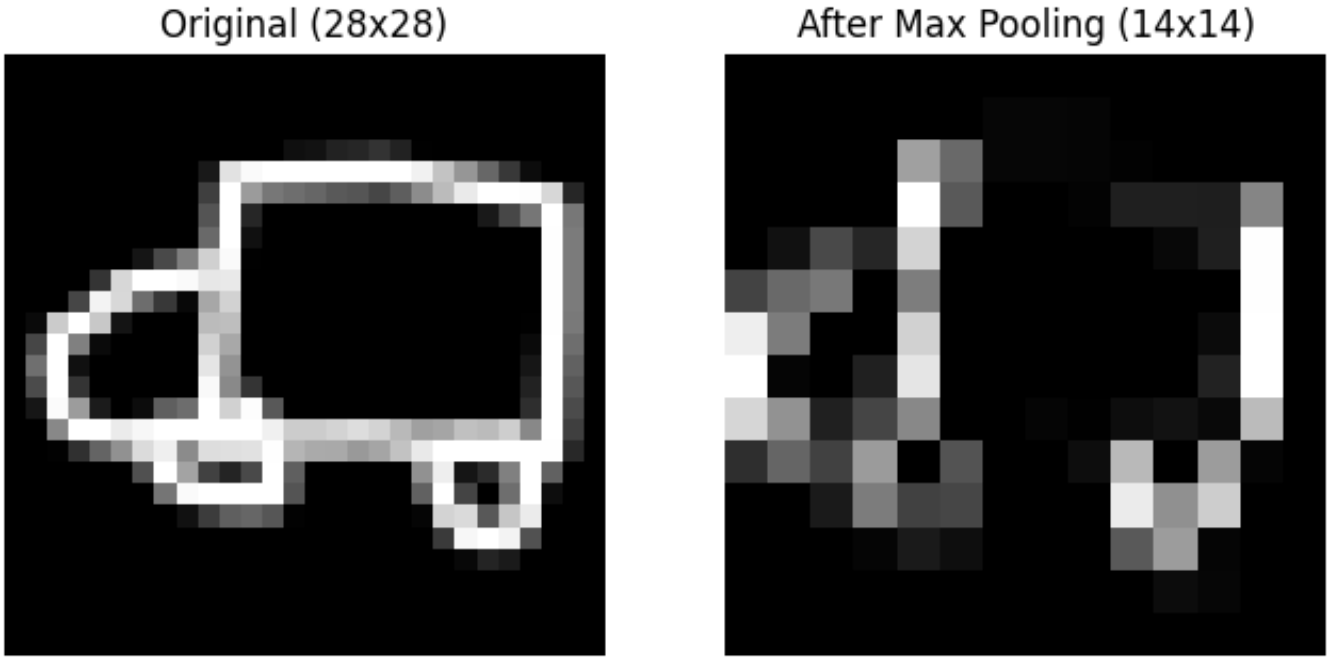


Figure 3.6: *Original image* (28×28) *and the most reduced image* (14×14)

Explanation. The 14×14 pooled output is the smallest form in which the data can still be viewed as an image; in the next step it becomes a list of numbers. It retains mainly the vertical edges of the van, the rear of the body, the front pillar and the sides of the wheels, using a quarter of the original number of pixels.

Results and Discussion

4.1 Progression of the Data through the Network

Table 4.1: *Shape and number of values after each stage*

Stage	Operation	Shape	No. of values
Input	Van #7, scaled to 0–1	28×28	784
Padding	Zero border, $P = 1$	30×30	900
Conv2D	3×3 vertical edge kernel, $S = 1$	28×28	784
ReLU	Negative values set to 0	28×28	784
Max pooling	2×2 window, $S = 2$	14×14	196
Flatten	Rows written out as one list	196	196
Fully connected	196×3 weights and 3 biases	3	3
Softmax	Scores converted to probabilities	3	3
Prediction		**van (47.4%)**	

The image is reduced from 784 pixel values to 196 feature values and finally to three probabilities. Every value along the way is produced by one of four elementary operations: *multiplication and addition* (convolution and fully connected layer), *discarding negatives* (ReLU), *taking a maximum* (pooling), and *exponentiation and division* (softmax).

4.2 Interpretation of the Output

- **Convolution** located the vertical edges of the van. Each vertical stroke produced positive values on its left side and negative values on its right side, with a largest magnitude of 3. Horizontal lines such as the roof produced values close to 0.
- **ReLU** removed every negative value, leaving one strong side of each vertical edge.
- **Max pooling** reduced 784 values to 196 while preserving the strongest responses and their approximate positions.
- **Flatten** arranged these 196 values as a single list without altering any of them.
- The **fully connected layer** combined the 196 values into three scores: van $= 0.36$, car $= 0.09$ and bus $= -0.71$.
- **Softmax** converted the scores into probabilities of 47.40%, 36.33% and 16.27%, giving a final prediction of **van**.

4.3 Random Weights and Trained Weights

The prediction "van" agrees with the image, but the agreement is **a matter of chance**. The network has never been shown what a van, a car or a bus looks like.

- **Random weights (this work).** The kernel was chosen by hand and the 588 fully connected weights are random numbers. Every calculation is carried out correctly, but the resulting scores carry no information about the image. With a different seed, for example `np.random.seed(1)`, the same van may be classified as a car or a bus. The network is in the state of a newly created model before any training.
- **Trained weights.** In a working CNN, the kernel values and the weights are *learned* from thousands of labelled images. After each prediction the error is measured by a loss function, and every weight is adjusted slightly in the direction that reduces it (backpropagation and gradient descent). After training, the kernels respond to patterns that are genuinely useful and the weights encode which patterns indicate each class. The correct class then receives the highest probability consistently, not by chance.

Table 4.2: *Comparison of random and trained weights*

Aspect	**Random weights (this work)**	**Trained weights**
Kernel	Hand-designed vertical edge detector	Learned from data
FC weights	Small random numbers with a fixed seed	Learned from data
Calculations	Correct	Identical calculations
Prediction	Effectively a guess	Meaningful and reliable
Effect of seed	May change the predicted class	Not applicable
Purpose	Understanding how a CNN computes	Recognising images

Training therefore does not change *how* a CNN computes its output; it changes only the *values* that enter those computations.

4.4 Limitations

- A single convolution layer with one kernel was used, so only vertical edges were detected. Practical CNNs use many kernels (horizontal, diagonal, curved) across several convolution layers.
- The weights were not trained, so the prediction does not reflect actual recognition.
- Only one image was processed, since the aim was to study the computations rather than to measure accuracy.

Conclusion

A single 28×28 drawing of a van from Google's Quick, Draw! dataset was passed through a complete CNN pipeline: padding, convolution with a 3×3 kernel, ReLU, max pooling, flattening, a fully connected layer and softmax. Each layer was implemented by hand in NumPy, its output matrix was printed and displayed, and selected values were verified manually.

The simulation shows that a CNN is built entirely from elementary arithmetic. The image was reduced from 784 pixel values to 196 feature values and finally to three probabilities, resulting in the prediction "van". Because the weights were not trained, this result demonstrates the procedure rather than real recognition. Training the same network on a large set of labelled drawings would leave every calculation unchanged while making its predictions reliable.

References

[1] Google Creative Lab, *The Quick, Draw! Dataset*.
https://quickdraw.withgoogle.com/data

[2] Google Creative Lab, *quickdraw-dataset*, GitHub repository.
https://github.com/googlecreativelab/quickdraw-dataset

[3] NumPy Developers, *NumPy Documentation*.
https://numpy.org/doc/

[4] Matplotlib Development Team, *Matplotlib Documentation*.
https://matplotlib.org/

[5] SciPy Developers, *scipy.special.softmax*.
https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.softmax.html

[6] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, Chapter 9: Convolutional Networks.